

Reviewer Pack — Minimal Grothendieck-Witt Class of Boolean Functions and Com...

Entry 0d8de3df0d8e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 06:40:06 UTC

Minimal Grothendieck-Witt Class of Boolean Functions and Communication Complexity Rank

Entry ID: 0d8de3df0d8e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 06:40:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Grothendieck-Witt theory) **Field B** (complexity object): Communication Complexity

Statement:

For any given n -ary boolean function f with communication complexity rank $R(f)$, the minimal Grothendieck-Witt class of f over the finite field $GF(2)$ is linearly correlated with $R(f)$, such that $|GW_class(f)| = \Theta(R(f))$.

Rationale (proposer's reasoning):

Grothendieck-Witt theory provides a framework for studying algebraic invariants of functions, which may reveal hidden structures in boolean functions. Communication complexity rank measures the communication cost between two parties to compute the function. If there is a linear correlation between these two invariants, it suggests that the structure captured by Grothendieck-Witt theory can be used to understand the complexity of communication.

Taxonomy category: GrothendieckWitt_to_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 00b85f72f7c2ea5b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the communication complexity rank $R(f)$ and the minimal Grothendieck-Witt class $|GW_class(f)|$ of n -ary boolean functions over $GF(2)$ exceeds 0.7, across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Grothendieck-Witt theory" AND "communication complexity" AND boolean functions - "finite field GF(2)" AND minimal Grothendieck-Witt class" AND rank of communication complexity" - "linear correlation" AND Grothendieck-Witt class" AND communication complexity rank for boolean functions

Top relevant hits considered: - [<http://arxiv.org/abs/1812.04171v2>] Finite unitary ring with minimal non-nilpotent group of units - [<http://arxiv.org/abs/2509.19568v1>] Knock-Knock: Black-Box, Platform-Agnostic DRAM Address-Mapping Reverse Engineering - [<http://arxiv.org/abs/1405.1018v5>] Generalized Fourier coefficients of multiplicative functions - [<http://arxiv.org/abs/astro-ph/0402107v1>] Bimodal distribution of the autocorrelation function in gamma-ray bursts - [<http://arxiv.org/abs/2312.13474v1>] Impact of tensor interactions and scalar mixing on covariant energy density functionals

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = int(math.log2(len(f)))
        return sum(f[i] != f[j] for i in range(2**n) for j in range(i+1, 2**n)) / (2**(n-1))

    def grothendieck_witt_class(f):
        n = int(math.log2(len(f)))
        e_sums = [0] * (n + 1)
        for i in range(2**n):
            for j in range(i+1, 2**n):
                if f[i] != f[j]:
                    e_sums[bin(i^j).count('1')] += 1
        return sum(e_sums) / (2**(n-1))

    def pearson_correlation(x, y):
        n = len(x)
        mean_x = sum(x) / n
        mean_y = sum(y) / n
```

```

cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
std_x = math.sqrt(sum((x[i] - mean_x)**2 for i in range(n)) / n)
std_y = math.sqrt(sum((y[i] - mean_y)**2 for i in range(n)) / n)
return cov_xy / (std_x * std_y) if std_x != 0 and std_y != 0 else 0

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    R_f = communication_complexity_rank(f)
    GW_class_f = grothendieck_witt_class(f)
    results.append((R_f, GW_class_f))

if not results:
    return {
        "metric_name": "Pearson Correlation",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

R_f_values, GW_class_f_values = zip(*results)
correlation_coefficient = pearson_correlation(R_f_values, GW_class_f_values)

return {
    "metric_name": "Pearson Correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for _, _ in results),
    "conjecture_holds": correlation_coefficient > 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE reason=no_results")
    else:
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
        elif any(not result["conjecture_holds"] for result in results):
            first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample='Pearson correlation < 0.7' first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the Pearson correlation coefficient could not be calculated. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24205
2	preregistration	ollama_remote	glm4:latest	0	0	17711
3	novelty	ollama_remote	glm4:latest	0	0	16686
4	novelty	ollama_remote	glm4:latest	0	0	9804
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15741
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	65131
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36608
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39035
9	judge	ollama_remote	glm4:latest	0	0	17456

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 242377 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0d8de3df0d8e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0d8de3df0d8e.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0d8de3df0d8e.tar.gz (if generated)